

CIE-2 Data Science

Theory

1 & 15. Difference Between / Comparison of Classification and Regression

This is the most fundamental concept. Both are types of supervised learning, but they predict different kinds of outcomes. Here is a direct comparison:

Basis of Comparison	Classification	Regression
Output Type	Predicts a discrete, categorical label (a class).	Predicts a continuous, numerical value .
Core Question	"Which category does this belong to?"	"How much or many ?"
Evaluation	Accuracy, Precision, Recall, F1-Score, Confusion Matrix.	Mean Absolute Error (MAE), Mean Squared Error (MSE), R-squared.
Examples	Is this email spam or not spam? Is this tumor malignant or benign? Which animal is in this image (cat, dog, bird)?	What will the temperature be tomorrow? What is the price of this house? How many units will this product sell?

In short, **classification separates data into distinct groups**, while **regression fits a line or curve to the data to predict a value**.

13. What is classification in machine learning? Name at least three classifiers.

Classification is a supervised machine learning task where the goal is to predict a predefined categorical label for a given piece of input data. The model learns from a labeled dataset and then uses that learning to assign new, unseen data to one of the learned categories.

Three common classifiers used in machine learning are:

1. **k-Nearest Neighbors (k-NN):** A simple, instance-based algorithm that classifies a new data point based on the majority class of its 'k' nearest neighbors.
 2. **Decision Tree:** A tree-like model that makes predictions by following a series of feature-based questions, leading to a leaf node that represents the final class label.
 3. **Logistic Regression:** Despite its name, this is a classification algorithm used to predict the probability that an input belongs to a particular category (typically used for binary classification).
-

3. What are the common types of classification, and in which scenarios are they used?

1. **Binary Classification:**
 - **What it is:** This is the simplest type, where there are only two possible outcomes or classes (e.g., Yes/No, 0/1, True/False).
 - **Scenario:** Used for problems like **email spam detection** (spam or not spam), **medical diagnosis** (disease or no disease), or **credit card fraud detection** (fraudulent or not fraudulent).
 2. **Multi-class Classification:**
 - **What it is:** This type involves three or more possible outcomes, where each sample can only belong to **one** class. The classes are mutually exclusive.
 - **Scenario:** Used for problems like **image recognition** (classifying an image as a cat, dog, or horse), **document classification** (tagging a news article as 'Sports', 'Politics', or 'Technology'), or **sentiment analysis** (classifying a review as 'positive', 'negative', or 'neutral').
 3. **Multi-label Classification:**
 - **What it is:** This type involves three or more possible outcomes, but a single sample can be assigned **multiple** labels simultaneously. The classes are not mutually exclusive.
 - **Scenario:** Used for problems like **movie genre classification** (a movie can be 'Action', 'Adventure', and 'Sci-Fi' all at once) or **tagging a blog post** with multiple relevant tags ('Data Science', 'Python', 'Machine Learning').
-

4. What are the main types of regression and how do they differ?

1. **Simple Linear Regression:**
 - **What it is:** Models the relationship between a single independent variable (X) and a single dependent variable (Y) by fitting a straight line.
 - **Equation:** $Y = \beta_0 + \beta_1 X + \epsilon$
 - **Difference:** It is the simplest form, used when you have only one feature to predict an outcome. For example, predicting a student's final exam score based only on the hours they studied.
2. **Multiple Linear Regression:**
 - **What it is:** Models the relationship between two or more independent variables (X1, X2, ...) and a single dependent variable (Y).

- **Equation:** $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n + \epsilon$
- **Difference:** It is an extension of simple linear regression. It's used when multiple factors influence the outcome. For example, predicting a house price based on its size, number of bedrooms, and location.

3. Polynomial Regression:

- **What it is:** Models a non-linear relationship between the independent and dependent variables by fitting a polynomial curve to the data.
- **Equation (e.g., 2nd degree):** $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$
- **Difference:** While the relationship it models is non-linear, it is still considered a type of linear regression because the equation is linear in terms of the coefficients (β). It's used when a straight line doesn't capture the underlying trend in the data, such as modeling the relationship between temperature and crop yield.

6. Identify some real-world applications of classification and regression techniques.

Real-World Classification Applications:

- **Healthcare:** Diagnosing whether a tumor is **malignant or benign**.
- **Finance:** Determining if a credit card transaction is **fraudulent or legitimate**.
- **Marketing:** Predicting if a customer will **churn (leave) or stay**.
- **Biology:** Classifying a plant into a known **species**.
- **Content Moderation:** Identifying online comments as **toxic or non-toxic**.

Real-World Regression Applications:

- **Real Estate:** Predicting the **selling price of a house** based on its features.
- **Finance:** Forecasting a company's **stock price** for the next day.
- **Retail:** Predicting the **number of units** a new product will sell.
- **Energy:** Forecasting the **electricity demand** for a city.
- **Automotive:** Predicting the **mileage (km/litre)** of a car based on its engine size and weight.

Classification implementation

2, 5 & 7. k-Nearest Neighbors (k-NN) and Nearest Neighbor Classifier

These questions are all related to the same algorithm. The "Nearest Neighbor" classifier is just a specific instance of k-NN where **k=1**.

How the k-NN Algorithm Works

k-NN is a simple, non-parametric algorithm that belongs to the "lazy learning" family. It's called "lazy" because it doesn't build a model during the training phase. It simply stores the entire dataset. The real work happens when a prediction is required for a new, unseen data point.

The steps for classification are:

1. **Choose the value of k:** Decide on the number of nearest neighbors to consider (e.g., $k=3$, $k=5$). This is a hyperparameter you must set.
2. **Calculate Distances:** For a new data point, calculate the distance between it and every single point in the training dataset. The most common distance metric is **Euclidean distance**.
 - Euclidean distance between two points $P(x_1, y_1)$ and $Q(x_2, y_2)$ is: $d = (x_2 - x_1)^2 + (y_2 - y_1)^2$
3. **Identify the k-Nearest Neighbors:** Select the 'k' data points from the training set that have the smallest calculated distances to the new point.
4. **Vote for the Class:** Among these 'k' neighbors, count the number of data points belonging to each class. The new data point is assigned the class that has the **majority vote**.

Example: Applying k-NN (k=3)

Let's use a small dataset to demonstrate. We have data on two features and a Class ('Red' or 'Blue').

Dataset:

Point	Feature 1 (X)	Feature 2 (Y)	Class
A	2	3	Red
B	3	4	Red
C	6	7	Blue
D	7	8	Blue

New Data Point to Classify: P (4, 5)

Let's set $k = 3$.

Step 1: Calculate Euclidean distance from P to all other points.

- $\text{Distance}(P, A) = (4-2)^2 + (5-3)^2 = 2^2 + 2^2 = 8 \approx 2.83$
- $\text{Distance}(P, B) = (4-3)^2 + (5-4)^2 = 1^2 + 1^2 = 2 \approx 1.41$
- $\text{Distance}(P, C) = (4-6)^2 + (5-7)^2 = (-2)^2 + (-2)^2 = 8 \approx 2.83$
- $\text{Distance}(P, D) = (4-7)^2 + (5-8)^2 = (-3)^2 + (-3)^2 = 18 \approx 4.24$

Step 2: Identify the $k=3$ nearest neighbors.

The three points with the smallest distances are:

1. **B** (distance 1.41)
2. **A** (distance 2.83)
3. **C** (distance 2.83)

Step 3: Vote for the class.

Let's look at the classes of these three neighbors:

- Neighbor B is **Red**.
- Neighbor A is **Red**.
- Neighbor C is **Blue**.

There are 2 votes for 'Red' and 1 vote for 'Blue'.

Conclusion: The majority class is **Red**. Therefore, the k-NN algorithm classifies the new point **P(4, 5)** as **'Red'**.

If we were using the **Nearest Neighbor classifier (k=1)**, we would only look at the single closest point, which is **B**. Since B's class is 'Red', the new point P would be classified as 'Red'.

8 & 16. Decision Trees

Explanation of Decision Trees

A **Decision Tree** is a supervised learning algorithm that is used for both classification and regression. It works by creating a tree-like model of decisions. The structure consists of:

- **Root Node:** The starting point of the tree, representing the entire dataset.
- **Internal Nodes:** Represent a test or question on a specific feature.
- **Branches:** Represent the outcome of the test (e.g., 'Yes'/'No' or 'High'/'Low').
- **Leaf Nodes:** Represent the final class label or decision.

To classify a new data point, you start at the root and traverse the path down the tree by answering the questions at each node until you reach a leaf node, which gives you the final prediction.

Example: Constructing a Simple Decision Tree

Let's use the classic "Play Tennis" problem.

Dataset:

Outlook Temp Humidity Wind Play Tennis
:--- :--- :--- :--- :---
Sunny Hot High Weak No
Sunny Hot High Strong No

| Overcast | Hot | High | Weak | Yes |

| Rainy | Mild | High | Weak | Yes |

| ... | ... | ... | ... | ... |

A decision tree built from this data might look like this:

[Is the Outlook Sunny, Overcast, or Rainy?]

|

+++ Sunny --- [Is Humidity High or Normal?]

| |

| +--- High --> [Play Tennis: No] (Leaf Node)

| +--- Normal --> [Play Tennis: Yes] (Leaf Node)

|

+++ Overcast -- [Play Tennis: Yes] (Leaf Node)

|

+++ Rainy --- [Is the Wind Weak or Strong?]

| |

 +--- Weak --> [Play Tennis: Yes] (Leaf Node)

 +--- Strong --> [Play Tennis: No] (Leaf Node)

How to use it: If you have a new day with Outlook=Sunny and Humidity=Normal, you would:

1. Start at the root: Is Outlook Sunny, Overcast, or Rainy? It's **Sunny**.
2. Follow the 'Sunny' branch. The next question is: Is Humidity High or Normal? It's **Normal**.
3. Follow the 'Normal' branch. You reach a leaf node.
4. **Prediction:** Play Tennis: Yes.

10. Simple Random Forest Approach

A **Random Forest** is an **ensemble learning** method. Instead of relying on a single decision tree, it builds *many* decision trees and combines their outputs to make a more accurate and stable prediction. It's like asking a diverse committee of experts for their opinion instead of just one.

How it works by combining multiple decision trees:

1. **Bagging (Bootstrap Aggregating):** The algorithm creates multiple random samples of the training data. For each new tree, a sample is drawn *with replacement* from the original dataset. This means some data points may be used multiple times in a single tree's training set, while others may not be used at all.
2. **Feature Randomness:** When building each tree, at every split point, the algorithm randomly selects a *subset* of features to consider for the best split, rather than evaluating all available features. This ensures the trees are different from one another.

3. **Build Multiple Trees:** A decision tree is trained on each of the random data samples with the random feature subsets.
4. **Voting for Classification:** To classify a new sample, it is passed down every single tree in the forest. Each tree gives a "vote" for a class. The final prediction is the class that receives the most votes.

By combining hundreds or thousands of de-correlated trees, the Random Forest reduces the risk of **overfitting** that a single deep decision tree might have, leading to better generalization on unseen data.

14. Program to Identify Native Speaker using a Decision Tree

To write this program, you would use a library like scikit-learn in Python. Here are the logical steps you would explain in an exam, which is more important than memorizing the exact code.

Objective: Use a decision tree to classify if a person is a 'native_speaker' or not based on features from a dataset.

Conceptual Steps / Pseudocode:

1. **Import necessary libraries:**
 - pandas to load and manage the data.
 - DecisionTreeClassifier from sklearn.tree to create the model.
 - train_test_split from sklearn.model_selection to separate data for training and testing.
2. **Load the Dataset:**
 - Read the 'reading_skills.csv' file into a pandas DataFrame.
 - `data = pandas.read_csv("reading_skills.csv")`
3. **Prepare the Data (Features and Target):**
 - Identify your feature columns (the inputs), for example: age, shoe_size, score. Let's call this X.
 - Identify your target column (the output you want to predict): native_speaker. Let's call this y.
 - `features = ['age', 'shoe_size', 'score']`
 - `X = data[features]`
 - `y = data['native_speaker']`
4. **Split the Data:**
 - Divide your dataset into a training set (to build the model) and a testing set (to evaluate its performance). A common split is 80% for training and 20% for testing.
 - `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)`
5. **Create and Train the Decision Tree Model:**
 - Create an instance of the DecisionTreeClassifier.
 - Train the model using the .fit() method on the training data.
 - `model = DecisionTreeClassifier()`
 - `model.fit(X_train, y_train)`
6. **Make Predictions:**
 - Use the trained model to make predictions on the unseen test data (X_test).
 - `predictions = model.predict(X_test)`
7. **Evaluate the Model (Optional but good practice):**

- Compare the model's predictions with the actual answers (y_{test}) to see how accurate it is using a metric like accuracy score.

This structured process is how you would apply the decision tree algorithm to a real-world dataset.

Regression implementation

9. Given a linear regression problem, formulate the regression equation and explain how it predicts new values.

Formulation of the Regression Equation

In a linear regression problem, we aim to model the relationship between one or more independent variables (features) and a single continuous dependent variable (target).

The equation for **Multiple Linear Regression** is formulated as:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$$

Where:

- Y is the **dependent variable** (the value we want to predict).
- $X_1, X_2, \dots, X_n$ are the **independent variables** (the features we use to make the prediction).
- β_0 is the **intercept** (or constant). It's the predicted value of Y when all independent variables are zero.
- $\beta_1, \beta_2, \dots, \beta_n$ are the **coefficients** for each independent variable. Each coefficient β_i represents the change in Y for a one-unit increase in the corresponding variable X_i , assuming all other variables are held constant.

The goal of training a linear regression model is to find the optimal values for the coefficients ($\beta_0, \beta_1, \dots$) that best fit the training data.

How It Predicts New Values

Once the model is trained and the coefficients are determined, making a prediction is straightforward. You simply substitute the feature values of the new, unseen data point into the equation.

Example:

Let's say we have a trained model to predict a car's Mileage (km/l) based on its Weight (kg) and Engine Size (cc).

After training, our model's equation is:

$$\text{Mileage} = 45.5 - 0.005 * \text{Weight} - 0.01 * \text{Engine_Size}$$

Now, we want to predict the mileage for a new car with the following features:

- Weight = 1200 kg
- Engine_Size = 1500 cc

Prediction Step: Plug the new values into the equation.

$$\text{Mileage} = 45.5 - 0.005 * (1200) - 0.01 * (1500)$$

$$\text{Mileage} = 45.5 - 6 - 15$$

$$\text{Mileage} = 24.5$$

Conclusion: The model predicts that this new car will have a mileage of **24.5 km/l**.

11. Apply logistic regression to a binary classification problem and interpret the output probabilities.

Applying Logistic Regression

Despite its name, **Logistic Regression** is a classification algorithm, not a regression one. It's primarily used for **binary classification** problems, where the outcome is one of two classes (e.g., Yes/No, True/False, 1/0).

The algorithm works by passing the weighted sum of inputs (similar to linear regression) through a **Sigmoid function**. The Sigmoid function squeezes any real number into a range between 0 and 1.

Sigmoid function: $P(Y=1)=1+e^{-z}$ (where z is the linear equation part)

This output is not a direct class label but a **probability**.

Interpreting the Output Probabilities

The output of a logistic regression model is the estimated probability that a given input sample belongs to the **positive class** (usually denoted by '1').

Example:

Let's say we've built a model to predict whether a customer will click on an ad (Click=1, No_Click=0) based on their age.

A new customer is 35 years old. We input this into our trained model, and it outputs the value **0.78**.

Interpretation:

- This output of 0.78 means there is a **78% probability** that this 35-year-old customer will click on the ad.
- The probability of the customer *not* clicking the ad is the complement: $1-0.78=0.22$, or **22%**.

To convert this probability into a definite class label, we use a **decision threshold**, which is typically set at **0.5**.

- If Probability ≥ 0.5 , we classify the instance as the positive class (1).
- If Probability < 0.5 , we classify the instance as the negative class (0).

In our example, since 0.78 is greater than 0.5, we would classify this customer as 'Click' (1).

12. Given a dataset, choose an appropriate regression model and justify your choice.

The choice of a regression model depends primarily on the relationship between the independent variables and the dependent variable, as well as the number of features.

Scenario 1: Predicting Student Grades based on Hours Studied

- **Dataset:** Contains two columns: Hours_Studied (independent) and Final_Grade (dependent). A scatter plot shows a clear, straight-line trend: as hours increase, the grade tends to increase proportionally.
- **Appropriate Model: Simple Linear Regression.**
- **Justification:** The problem involves a single independent variable and a single dependent variable. Visual inspection of the data suggests a **linear relationship**. Simple Linear Regression is the most direct and computationally efficient model for capturing this kind of relationship.

Scenario 2: Predicting a House Price

- **Dataset:** Contains multiple columns: Square_Footage, Num_Bedrooms, Num_Bathrooms, House_Age, and the target Price. It's logical to assume all these factors contribute to the price.
- **Appropriate Model: Multiple Linear Regression.**
- **Justification:** The price is influenced by **multiple independent variables**. Multiple Linear Regression is designed specifically for this purpose, allowing the model to determine the individual contribution and weight of each feature (Square_Footage, Num_Bedrooms, etc.) to the final predicted Price.

Scenario 3: Predicting Crop Yield based on Rainfall

- **Dataset:** Contains two columns: Rainfall and Crop_Yield. A scatter plot shows that yield increases with rainfall up to a certain point, after which excessive rainfall causes the yield to decrease. The plot shows a distinct **curve** (an inverted 'U' shape).
 - **Appropriate Model: Polynomial Regression.**
 - **Justification:** A straight line (linear regression) would be a very poor fit for this data and would result in large errors. The relationship is clearly **non-linear**. A Polynomial Regression model (e.g., of the 2nd degree) can fit a curve to the data, accurately capturing the trend where the yield first rises and then falls.
-